2026

Arquitectura Cliente-Servidor y Patrones de Desarrollo Web

IVÁN MARÍN VIADERO

DESARROLLO WEB EN ENTORNO CLIENTE

Índice

Introducción ... 2
Arquitectura cliente-servidor ... 3
Comparación con una aplicación monolítica local ... 3
Ventajas e inconvenientes ... 3
Componentes principales ... 4
Cliente ... 4
Servidor y servicios ... 4
Responsabilidad y seguridad ... 4
Comunicación HTTP y HTTPS ... 5
Métodos habituales ... 5
Ejemplo práctico ... 6
Arquitectura en capas ... 7
Modelo-Vista-Controlador ... 8
Frameworks relacionados con MVC ... 8
REST y API REST ... 9
Principios fundamentales ... 9
Diseño de recursos para una API imaginaria ... 9
SPA (Single Page Application) ... 10
Funcionamiento ... 10
Ventajas e inconvenientes ... 10
SSR (Server Side Rendering) ... 10
Ventajas e inconvenientes ... 10
Arquitectura Jamstack ... 11
Características y casos de uso ... 11
Frontend ... 11
Tecnologías habituales ... 11
Backend ... 12
Tecnologías habituales ... 12
Comparativa entre frontend y backend ... 12
Visión conjunta ... 12
Caso de estudio (YouTube) ... 13
Alcance y arquitectura ... 13
Flujo simplificado ... 13
Análisis técnico ... 14
Diseño de aplicación propia (NeedIt) ... 15
Diagrama de arquitectura ... 15
API Rest propuesta ... 16
Seguridad y evolución ... 16
Conclusiones ... 17
Bibliografía ... 18

Introducción

La mayoría de las aplicaciones web actuales distribuyen sus tareas entre dispositivos de usuario, servidores de aplicaciones y sistemas de almacenamiento. Esta distribución permite utilizar un mismo servicio desde un navegador, un teléfono móvil u otros clientes, pero introduce decisiones técnicas relacionadas con el rendimiento, la seguridad, el mantenimiento y la escalabilidad.

Este trabajo analiza los conceptos de arquitectura cliente-servidor, comunicación HTTP, diseño por capas, patrón Modelo-Vista-Controlador (MVC) y uso de API REST. También compara alternativas para construir interfaces, como SPA, renderizado en servidor (SSR) y Jamstack. Finalmente, aplica estas ideas a un servicio conocido y a una propuesta propia.

Objetivos:

- Identificar responsabilidades y flujos de comunicación de cada componente de una aplicación web.
- Comprender las diferencias entre patrones de organización del código y estrategias de renderizado.
- Interpretar técnicamente una plataforma web real, diferenciando hechos documentados de hipótesis.
- Diseñar y justificar una arquitectura viable para una aplicación de alquiler entre particulares.

Alcance y metodología

La investigación se ha apoyado en documentación técnica oficial: MDN, especificaciones y fuentes de los frameworks, Google for Developers y la definición original de REST de Roy Fielding. Los textos son una síntesis propia. Los esquemas son elaboraciones originales. El análisis de YouTube no pretende describir código o infraestructura privada a la que no se tiene acceso.

Arquitectura cliente-servidor

La arquitectura cliente-servidor es un modelo distribuido en el que un cliente solicita un recurso o servicio y un servidor procesa la petición y devuelve una respuesta. En la web, el cliente suele ser un navegador y el intercambio se realiza mediante HTTP o HTTPS. El servidor puede entregar archivos o solicitar datos a otros componentes para generar contenido dinámico. *[1, 2]*

Comparación con una aplicación monolítica local

Una aplicación monolítica local integra habitualmente interfaz, lógica y almacenamiento en un mismo equipo y se ejecuta sin necesitar un servidor remoto para sus funciones principales. La comparación se refiere al despliegue local frente al modelo distribuido: “monolítica” también puede describir un backend web organizado como una sola unidad. Por tanto, cliente-servidor y monolito no son categorías opuestas. *[2]*

Aspecto	Cliente-servidor web	Monolítica local
Ejecución	Cliente y servicios en sistemas separados	Mayor parte del sistema en un equipo
Datos	Habitualmente centralizados	Pueden almacenarse localmente
Acceso	Desde múltiples dispositivos con red	Normalmente desde el equipo instalado
Actualización	Servidor actualizado centralmente	Requiere actualizar cada instalación

Ventajas e inconvenientes

Ventajas:

- Centralización de datos.
- Acceso desde varios dispositivos.
- Actualización del servicio sin reinstalar el cliente.
- Posibilidad de ampliar la infraestructura cuando crece el número de usuarios.

Inconvenientes:

- Dependencia de la red y de la disponibilidad del servidor.
- Costes de alojamiento y supervisión.
- Superficie de ataque más amplia.
- Necesidad de diseñar bien la comunicación entre componentes.

Ejemplo: una tienda web permite consultar el mismo catálogo desde un ordenador y un móvil. Si cambia el precio en el servidor, ambos clientes obtendrán el dato actualizado en su siguiente consulta.

Componentes principales

Cliente

Navegador web: interpreta HTML y CSS, ejecuta JavaScript, muestra la interfaz, gestiona interacciones y puede enviar peticiones HTTP. Algunos ejemplos son Google Chrome, Mozilla Firefox o Microsoft Edge. *[1]*

Aplicación móvil: permite ofrecer interfaces y funciones adaptadas a dispositivos móviles. Puede comunicarse con el mismo backend mediante una API, aunque su interfaz no sea una página web.

Aplicación de escritorio: programa ejecutado en Windows, macOS o Linux. Puede utilizar un servicio remoto para sincronizar cuentas, documentos y otras operaciones.

Servidor y servicios

Servidor web: recibe conexiones HTTP/HTTPS, entrega recursos estáticos y, según su configuración, redirige peticiones a un proceso de aplicación. Algunos ejemplos son Nginx o Apache HTTP Server. *[4]*

Servidor de aplicaciones: ejecuta la lógica del backend: valida datos, comprueba permisos, implementa reglas y construye las respuestas. Puede utilizar Java con Spring Boot, PHP, C# o Node.js. *[3, 5]*

Servidor de base de datos: conserva información persistente: usuarios, productos, pedidos, etc. Puede ejecutarse en una máquina independiente o compartir infraestructura en proyectos pequeños. Algunos ejemplos de tecnologías de almacenamiento son MySQL, PostgreSQL o MongoDB. *[3]*

Responsabilidad y seguridad

El cliente no debe conectar directamente con la base de datos ni decidir por sí solo si un pago o una reserva son válidos. Es el backend quien verifica las operaciones y accede a los datos con permisos limitados. Esta separación disminuye el riesgo de exponer contraseñas o reglas sensibles. *[3]*

Comunicación HTTP y HTTPS

HTTP (Hypertext Transfer Protocol) es el protocolo de aplicación que permite intercambiar solicitudes y respuestas entre clientes y servidores web. HTTPS utiliza HTTP protegido mediante TLS (Transport Layer Security), de modo que aporta cifrado y autenticación del servidor. En ambos casos, el intercambio se compone de peticiones y respuesta. *[1, 6]*

Una petición suele incluir método, ruta o URI, cabeceras y, cuando corresponde, cuerpo. Una respuesta devuelve un código de estado, cabeceras y opcionalmente datos, por ejemplo, HTML o JSON. Un “200 OK” indica éxito; “404 Not Found”, recurso inexistente; y “500 Internal Server Error”, fallo del servidor.

Métodos habituales

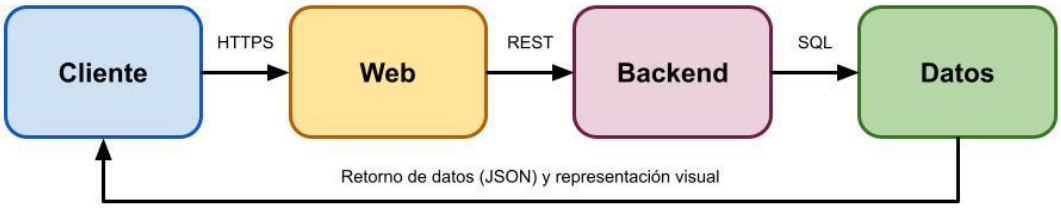
Método	**Finalidad**	**Ejemplo en tienda**
GET	Consultar una representación sin modificarla	GET /api/productos/42
POST	Enviar datos para crear o ejecutar una acción	POST /api/pedidos
PUT	Sustituir el estado del recurso objetivo	PUT /api/productos/42
DELETE	Solicitar la eliminación de un recurso	DELETE /api/productos/42
PATCH	Aplicar una modificación parcial	PATCH /api/productos/42

Fuente: MDN Web Docs [7]

Ejemplo breve: al visitar la ficha de un artículo, el navegador envía “GET /api/productos/42”. El servidor responde con estado “200” y datos JSON, por ejemplo {"id": 42, "nombre": "Taladro", "precio": 35}. El frontend utiliza esos datos para mostrar el nombre y el precio sin escribirlos de forma fija en el código.

Ejemplo práctico

Se analiza una tienda online imaginaria. El usuario selecciona un producto para ver su descripción, precio y disponibilidad. En el siguiente esquema se representa un posible sistema con frontend y backend separados.



Esquema del sistema de front y back. Fuente: elaboración propia

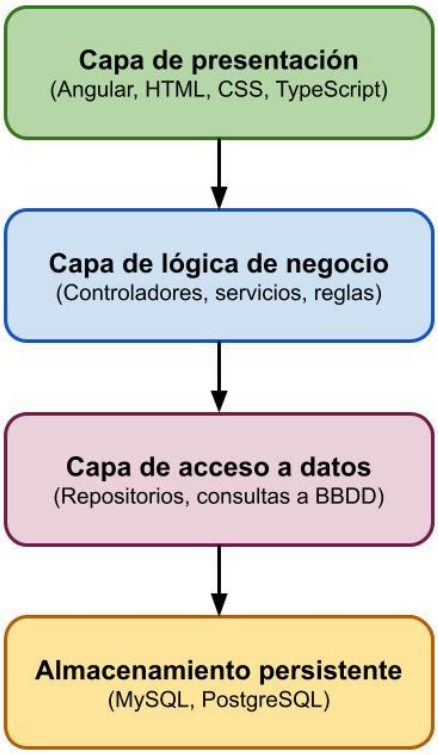
Secuencia de la operación:

1. El usuario pulsa en “Ver producto”. El navegador abre una ruta de la tienda y muestra la vista correspondiente.
2. El cliente solicita “GET /api/productos/42” al servidor, normalmente por HTTPS.
3. El servidor web entrega la petición al backend, que identifica el recurso solicitado.
4. El servicio de productos consulta la base de datos y recupera nombre, precio y existencias.
5. El backend valida el resultado y devuelve un JSON con código HTTP “200” (o “404” si el artículo no existe).
6. El frontend actualiza la vista con los datos. El usuario visualiza la información sin conocer los detalles internos del almacenamiento.

Si el usuario realiza una compra, el proceso cambia: el backend debe validar el stock, calcular importes y registrar el pedido, entre otras cosas.

Arquitectura en capas

Una arquitectura en capas organiza el software en función de sus responsabilidades. Cada capa atienda una clase de problemas y se comunica con capas definidas, evitando mezclar componentes visuales, reglas del negocio y consultas de datos. Su objetivo es facilitar cambios, pruebas y comprensión del sistema. *[2]*



Arquitectura en capas. Fuente: elaboración propia

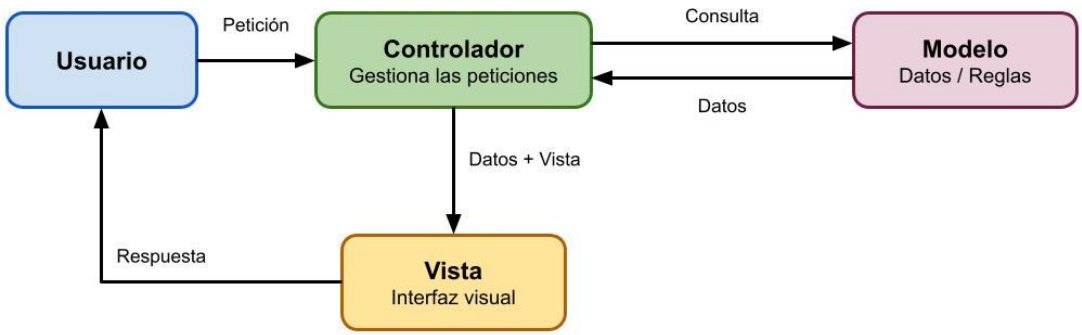
Capas habituales

- Presentación: muestra la aplicación y recoge acciones del usuario. Utiliza tecnologías como HTML, CSS, JavaScript, Angular, React o Vue.
- Lógica de negocio: ejecuta reglas de dominio como autorizaciones, precios, descuentos o límites de reserva. Puede implementarse con Java, PHP, C# o JavaScript ejecutado con Node.js.
- Acceso a datos: encapsula operaciones de almacenamiento y consulta. Por ejemplo, repositorios JPA de Java que usan PostgreSQL o MySQL. *[8]*

Este diseño facilita reutilizar reglas, sustituir tecnologías de persistencia y probar servicios sin depender de la interfaz. A cambio, introduce archivos, interfaces y llamadas adicionales.

Modelo-Vista-Controlador

El patrón MVC separa una aplicación en modelo, vista y controlador. Su propósito es evitar que toda la lógica se concentre en una única clase o página. El funcionamiento exacto varía entre frameworks y no exige que frontend y backend estén escritos con la misma tecnología. *[9]*



Patrón Modelo-Vista-Controlador. Fuente: elaboración propia

Modelo: representa los datos y el estado del dominio y puede incluir reglas de negocio o delegar el acceso a repositorios.

Vista: presenta información e interacciones como fichas, formularios o listados. En MVC tradicional, suele generarse mediante plantillas HTML.

Controlador: recibe las solicitudes, comprueba qué operación corresponde, se apoya en el modelo y determina la respuesta apropiada.

Frameworks relacionados con MVC

Framework	Lenguaje/Plataforma	Aplicación del patrón
Laravel	PHP	Controladores, modelos Eloquent y vistas Blade *[10, 11, 12]*
Spring MVC	Java/Spring	Controladores y DispatcherServlet. Puede devolver vistas o JSON *[5, 13]*
ASP.NET Core MVC	C#/.NET	Modelos, controladores y vistas Razor *[9]*
Django	Python	Variante MTV: Model, Template y View *[14]*

REST y API REST

REST (Representational State Transfer) es un estilo arquitectónico definido por Roy Fielding para sistemas distribuidos de hipermedia. Una API REST expone recursos mediante una interfaz coherente, habitualmente usando HTTP, URI y representaciones como JSON. *[15]*

Principios fundamentales

Cliente-servidor: separación entre la interacción del usuario y la gestión de datos y recursos.

Stateless (sin estado de sesión en el servidor): cada petición contiene la información suficiente para interpretarla sin depender de una sesión previa almacenada en el servidor.

Caché: las respuestas indican si son reutilizables para reducir solicitudes innecesarias y mejorar tiempos de respuesta.

Interfaz uniforme: los recursos se identifican de manera consistente y se manipulan mediante representaciones y métodos bien definidos.

Recursos con URI: cada recurso tiene una identificación estable, como por ejemplo "api/productos/42". Además, REST incluye la restricción de sistema en capas y, opcionalmente, código bajo demanda. *[15]*

Diseño de recursos para una API imaginaria

Operación	Ejemplo	Resultado esperado
Consultar catálogo	GET /api/productos	Lista JSON de productos
Ver artículo	GET /api/productos/42	Detalle JSON del producto
Publicar artículo	POST /api/productos	Nuevo recurso ("201 Created")
Editar precio	PATCH /api/productos/42	Datos modificados
Eliminar anuncio	DELETE /api/productos/42	Eliminación ("204 No Content")

SPA (Single Page Application)

Una SPA carga inicialmente una estructura HTML y código JavaScript que gestiona las vistas. Al navegar por la aplicación, un enrutador del cliente actualiza el contenido sin recargar toda la página. Para conseguir datos nuevos puede consultar una API HTTP. *[16]*

Funcionamiento

1- Descarga inicial de HTML, CSS y JavaScript.
2- Inicio de la aplicación en el navegador.
3- Navegación mediante rutas del cliente.
4- Llamadas a servicios para obtener o actualizar datos. Angular, React y Vue permiten construir interfaces de este tipo, aunque también admiten otras estrategias. *[16, 17, 18]*

Ventajas e inconvenientes

Ventajas:

- Interacciones ágiles una vez cargada la página.
- Reutilización de componentes.
- Separación clara entre cliente y API.

Inconvenientes:

- Descarga inicial potencialmente elevada.
- Mayor dependencia del JavaScript.
- Necesidad de atender con cuidado la indexación y la accesibilidad

SSR (Server Side Rendering)

El renderizado del lado del servidor genera HTML en el servidor para enviarlo ya construido al navegador. En aplicaciones modernas puede combinarse con hidratación, proceso mediante el que JavaScript aporta interactividad al HTML recibido. *[19]*

Ventajas e inconvenientes

Ventajas:

- Puede mejorar la rapidez con la que aparece contenido útil.
- Facilita que algunos rastreadores interpreten la página.
- Permite preparar información en el servidor antes de enviarla.

Inconvenientes:

- Añade trabajo de renderizado al servidor, consumo de recursos y complejidad de despliegue.
- Es necesario gestionar la hidratación cuando la página es interactiva.

Algunos frameworks representativos son Next.js (ecosistema React) o Nuxt.js (ecosistema Vue). *[19, 20]*

Arquitectura Jamstack

Jamstack propone separar la capa de presentación de los servicios de datos y de la lógica de negocio. Su principio más representativo es generar previamente páginas y recursos estáticos y servirlos mediante una CDN (Content Delivery Network), añadiendo interactividad mediante JavaScript y API cuando sea necesaria.

Características y casos de uso

- **Prerrenderizado:** páginas listas antes de la visita del usuario, sin necesidad de generarlas desde cero en cada acceso.
- **Desacoplamiento:** el frontend puede desplegarse separadamente del contenido, autenticación u otros servicios.
- **Content Delivery Network (CDN):** distribución geográfica de recursos para mejorar la latencia y reducir la carga del origen.

La arquitectura Jamstack se utiliza frecuentemente en páginas informativas, portfolios, documentación y tiendas con mucho contenido editorial. *[21]* Para datos muy personalizados y continuamente cambiantes requiere combinar páginas estáticas con APIs, renderizado dinámico u otras soluciones.

Frontend

El frontend reúne los elementos con los que interactúa el usuario: estructura, diseño, navegación, formularios y comportamiento en el navegador. Es responsable de presentar datos con claridad, gestionar estados de interfaz y ofrecer una experiencia adaptable y accesible. *[3]*

Tecnologías habituales

- **HTML:** estructura semántica del contenido y formularios.
- **CSS:** presentación visual, diseño responsivo y distribución de elementos.
- **JavaScript y TypeScript:** interacciones, validaciones en la interfaz y comunicación con APIs.
- **Angular, React y Vue:** desarrollo basado en componentes. Angular es un framework, React es una biblioteca de interfaz y Vue es un framework progresivo. *[16, 17, 18]*

Nota: una validación en el frontend puede mejorar la usabilidad, pero nunca debe sustituir las comprobacione de seguridad y consistencia del backend.

Backend

El backend procesa operaciones y protege los datos que no deben depender de las decisiones del navegador. Entre sus responsabilidades se encuentran la autenticación, autorización, reglas comerciales, validación, acceso a la base de datos e integración con otros servicios. *[3, 5]*

Tecnologías habituales

Algunas combinaciones habituales de tecnologías usadas en backend son:

- Java con Spring Boot
- PHP con Laravel
- C# con ASP.NET Core
- JS/TS con Node.js
- Python con Django

Comparativa entre frontend y backend

Aspecto	**Frontend**	**Backend**
Objetivo	Presentar información y recoger interacciones	Ejecutar reglas, validad y gestionar información
Tecnologías	HTML, CSS, JS, TS, Angular, React, Vue	Java, Spring Boot, PHP, Laravel, C#, Node.js, Django
Ejemplo	Mostrar una ficha y un botón “Alquilar”	Validar fechas, bloquear reservas incompatibles y guardar la operación
Ejecución	Principalmente en el navegador	En un servidor o entorno de ejecución
Seguridad	No guarda secretos ni impone permisos definitivos	Verifica permisos y protege los datos

Visión conjunta

En una aplicación web real, el frontend y el backend se complementan entre ellos. Una operación completa puede comenzar al clicar un elemento en Angular, continuar haciendo “POST” a una API de Spring Boot, realizar una consulta de PostgreSQL y finalizar con una confirmación visual en el contenido del navegador.

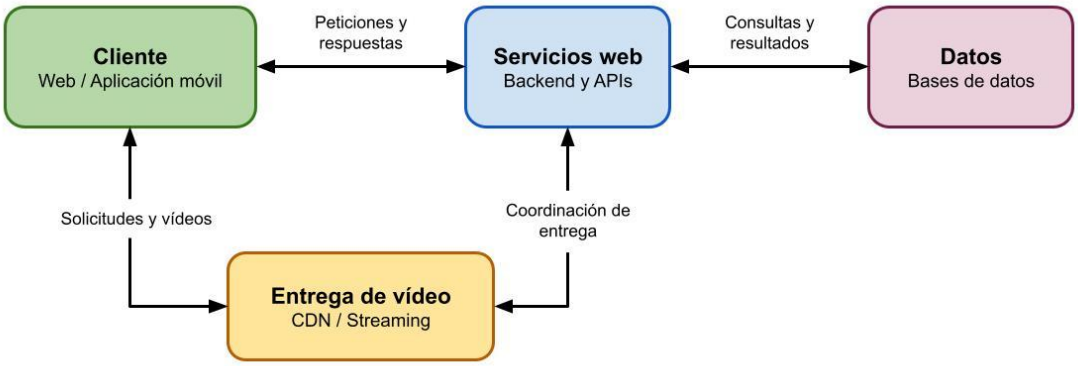
Además, la separación entre front y back también ofrece independencia, ya que, por ejemplo, una API podría ser utilizada tanto por una web como por una aplicación móvil, sin necesidad de duplicar la lógica de negocio.

Caso de estudio (YouTube)

Alcance y arquitectura

YouTube es una plataforma web que permite buscar, reproducir y publicar vídeos, gestionar canales, crear listas de reproducción y realizar diversas interacciones. Puede usarse desde navegadores y aplicaciones móviles, lo cual encaja con una arquitectura cliente-servidor: distintos clientes solicitan servicios y reciben respuestas en red.

Existe una API oficial proporcionada y desarrollada por Google llamada YouTube Data API, que expone datos y recursos como vídeos, listas y canales para aplicaciones autorizadas. *[22]* También existe una IFrame Player API, que permite incorporar un reproductor de YouTube en sitios web externos y controlarlo con JavaScript. *[23]*



Arquitectura simplificada de YouTube. Fuente: elaboración propia

Flujo simplificado

A continuación, se propone un ejemplo simplificado de un flujo de reproducción en YouTube:

1. En primer lugar, un usuario abre un vídeo desde la web o la aplicación móvil de YouTube.
2. El cliente obtiene información de la página y del vídeo, como el título, miniatura, duración, tiempo transcurrido desde la subida y el número de reproducciones.
3. El reproductor solicita el contenido multimedia a los servidores de YouTube y comienza a reproducirlo directamente en la página, descargando los datos necesarios durante la reproducción.
4. En dicha página, la interfaz permite acciones posteriores, como pausar y reanudar el vídeo, comentar o seleccionar otro vídeo.

Análisis técnico

MVC y posibles patrones

YouTube no se identifica públicamente como una aplicación basada principalmente en el patrón Modelo-Vista-Controlador (MVC). Su funcionamiento requiere numerosos componentes para gestionar usuarios, vídeos, búsquedas, comentarios y reproducción de contenido. Además, se ha documentado el uso de tecnologías como Polymer para desarrollar interfaces mediante componentes reutilizables. *[24]* Aunque algunas partes internas podrían seguir una organización similar a MVC, no hay información pública suficiente para confirmar que este sea el patrón principal de la plataforma.

API y separación front/back

La API pública de datos de YouTube permite consultar ciertos recursos y gestionar operaciones autorizadas mediante credenciales, mientras que la API del reproductor controla reproducción y eventos en sitios web externos. *[22, 23]* El frontend visible para el usuario se encarga del buscador, menús y reproductor, mientras que el backend debe gestionar la información y las operaciones que no pueden confiarse al navegador, como permisos de cuentas o persistencia del contenido.

Posible uso de microservicios

YouTube se apoya en la infraestructura distribuida de Google, que utiliza microservicios para dividir las aplicaciones en servicios independientes. Google ha documentado esta arquitectura y ha mencionado el procesamiento de vídeos de YouTube como una de las tareas que se ejecutan en su infraestructura. *[25]* Esta organización permite distribuir el trabajo entre distintos servicios y ampliar los recursos de cada uno según las necesidades. En una plataforma como YouTube, esto resulta especialmente útil debido a la gran cantidad de usuarios, reproducciones y vídeos que se deben gestionar. Sin embargo, no se conoce públicamente la distribución completa de los microservicios internos de YouTube ni cuántos utiliza exactamente.

Tecnologías utilizadas por YouTube

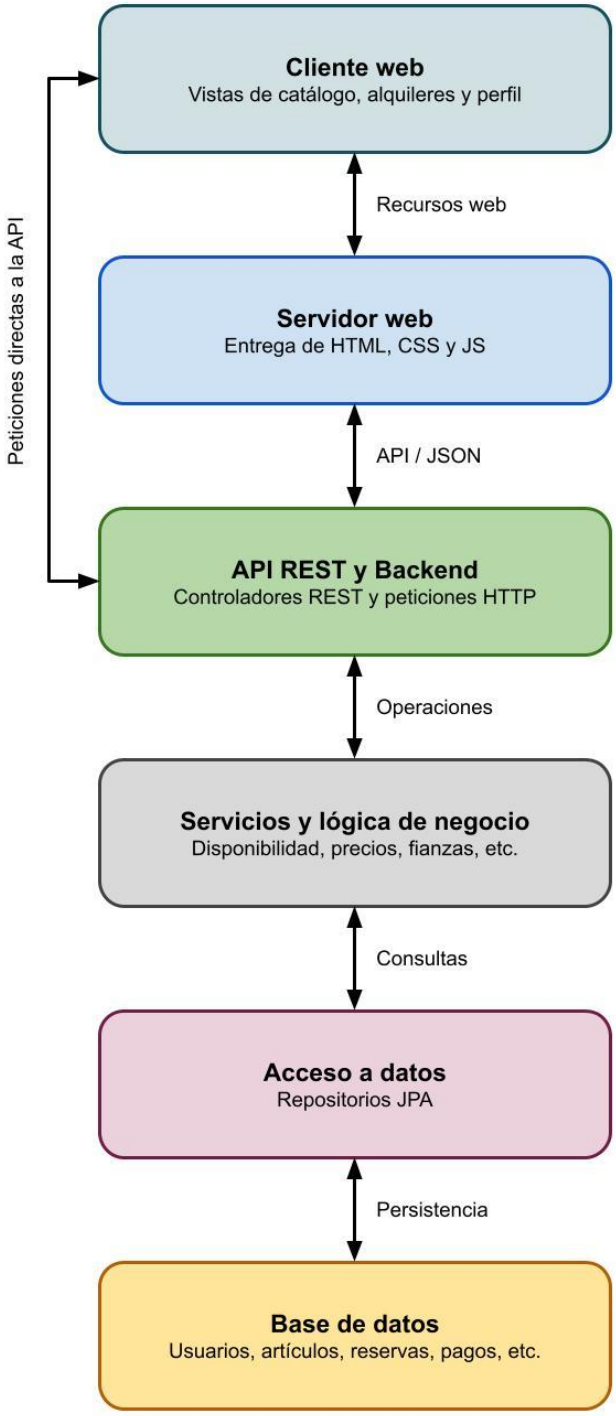
Elemento	Tecnología	Función
Frontend	HTML, CSS, JavaScript	Desarrollo de la interfaz web y sus componentes
Backend e infraestructura	Servicios distribuidos *[25]*	Gestión de usuarios, vídeos y otras operaciones
Bases de datos	Google Spanner y Bigtable *[26, 27]*	Almacenamiento y procesamiento de datos
Distribución	Google Global Cache (GGC) *[28]*	Distribución de contenido desde servidores cercanos

YouTube utiliza diferentes tecnologías para gestionar su infraestructura. Entre ellas destacan Spanner y Bigtable, cuyo uso está documentado por Google. Históricamente, también utilizó MySQL junto con Vitess, un sistema desarrollado originalmente por YouTube y escrito principalmente en Go para facilitar el escalado de sus bases de datos. *[29, 30]* Sin embargo, no se ha podido confirmar que Vitess siga desempeñando la misma función actualmente.

Diseño de aplicación propia (NeedIt)

NeedIt se plantea como una plataforma web para alquilar objetos entre particulares: herramientas, máquinas de coser, impresoras 3D y otros artículos de uso ocasional. El usuario puede consultar anuncios cercanos, elegir fechas y solicitar una reserva.

Diagrama de arquitectura



Arquitectura propuesta para NeedIt. Fuente: elaboración propia

Justificación de decisiones

Patrón elegido: se ha elegido una arquitectura cliente-servidor dividida en capas y backend monolítico modular. Angular se encargaría de la interfaz visual, mientras que el backend de Spring Boot organizaría sus componentes en controladores, servicios y repositorios. Para una primera versión no sería necesario utilizar microservicios, ya que aumentarían la complejidad del desarrollo y del mantenimiento de la aplicación sin aportar ventajas importantes en esta fase.

Cliente: se propone el uso de Angular, ya que permite crear una SPA con componentes reutilizables: catálogo, filtros, perfiles y reservas. Su enrutador mantiene la navegación entre pantallas sin recargar la página completa. *[16]*

Servidor web: se propone usar Nginx, ya que entrega los archivos compilados de Angular y reenvía las peticiones de la API al proceso de Spring Boot, además de que podría centralizar TLS y ciertas reglas de acceso. *[4, 6]*

Backend y REST: se propone Spring Boot con controladores REST en Java. *[5]* Los servicios verificarían que el artículo esté disponible, que las fechas sean válidas, que el usuario tenga permisos y que la fianza aplicada sea correcta.

Base de datos: se propone usar PostgreSQL para registrar usuarios, anuncios, disponibilidad, reservas y pagos relacionados. Un modelo relacional facilitaría restricciones y transacciones, especialmente para evitar dobles reservas. *[31]*

API Rest propuesta

Endpoint	Descripción
GET /api/articulos	Obtener anuncios y filtrar por categoría o proximidad
GET /api/articulos/{id}	Consultar la ficha y condiciones de alquiler de un producto
POST /api/artículos	Publicar un nuevo anuncio
POST /api/reservas	Solicitar alquiler para unas fechas concretas
PATCH /api/reservas/{id}	Modificar el estado de una reserva tras la validación del servidor

Seguridad y evolución

Para garantizar la seguridad de NeedIt, las operaciones importantes, como iniciar sesión, publicar anuncios o realizar reservas, se validarían desde el backend. También se utilizaría HTTPS para proteger la comunicación entre el cliente y el servidor. *[6]* En cuanto a los alquileres, el backend comprobaría la disponibilidad de los artículos durante las fechas seleccionadas y utilizaría mecanismos de control de concurrencia para evitar que dos usuarios pudieran confirmar reservas incompatibles. Para gestionar los pagos y las fianzas, sería necesario integrar un servicio de pagos externo.

Si la plataforma crece y aumenta el número de usuarios, podrían incorporarse mejoras para optimizar su funcionamiento, como un sistema de caché para acelerar las búsquedas o servicios independientes para gestionar las imágenes, los pagos y las reservas.

Conclusiones

La investigación muestra que, a la hora de construir una web, el aspecto decisivo es distribuir correctamente las responsabilidades y definir cómo colaboran los componentes. La arquitectura cliente-servidor establece el intercambio de solicitudes y respuestas, el modelo en capas reduce dependencias, y la arquitectura MVC organiza partes de la aplicación en torno al modelo, la vista y el controlador.

HTTP proporciona un mecanismo común para comunicar clientes y servidores. Una API REST puede aprovecharlo para identificar recursos y ofrecer operaciones coherentes, pero no debe confundirse la presencia de JSON con una aplicación completa del estilo REST. Del mismo modo, SPA, SSR y Jamstack no son patrones intercambiables: responden principalmente a preguntas sobre dónde y cuándo se genera el contenido, y cómo se entrega al usuario.

El estudio de la plataforma de YouTube también evidencia un límite importante: es posible explicar su naturaleza cliente-servidor y citar API públicas sin conocer su arquitectura privada. Presentar con claridad qué está documentado y qué es una hipótesis mejora la calidad del análisis, especialmente en sistemas de gran escala.

Para NeedIt (aplicación propia), considero adecuada una primera versión con Angular, Spring Boot y PostgreSQL. Esta combinación mantiene separadas la interfaz, las reglas de alquiler y los datos, y permite reutilizar la API desde futuros clientes. He descartado empezar con microservicios porque el tamaño inicial del proyecto no justifica sus costes de administración.

En definitiva, no existe una arquitectura universalmente superior: una buena elección depende de la complejidad, del equipo, del tráfico esperado y de las necesidades de seguridad. La solución más sólida suele ser la que resuelve el problema actual con claridad y deja abierta una evolución razonable.

Bibliografía

1. Descripción general de cliente-servidor – MDN: https://developer.mozilla.org/es/docs/Learn_web_development/Extensions/Server-side/First_steps/Client-Server_overview
2. Network-based Architectural Styles – Roy Fielding: https://ics.uci.edu/~fielding/pubs/dissertation/net_arch_styles.htm
3. Introducción al lado del servidor – MDN: https://developer.mozilla.org/es/docs/Learn_web_development/Extensions/Server-side/First_steps/Introduction
4. Beginner's Guide – NGINX: https://nginx.org/en/docs/beginners_guide.html
5. Servlet Web Applications – Spring: https://docs.spring.io/spring-boot/reference/web/servlet.html
6. Transport Layer Security (TLS) – MDN: https://developer.mozilla.org/en-US/docs/Web/Security/Defenses/Transport_Layer_Security
7. HTTP request methods – MDN: https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Methods
8. Core concepts – Spring: https://docs.spring.io/spring-data/jpa/reference/repositories/core-concepts.html
9. Información general de ASP.NET Core MVC – Microsoft Learn: https://learn.microsoft.com/es-es/aspnet/core/mvc/overview?view=aspnetcore-10.0
10. Controllers – Laravel: https://laravel.com/framework/docs/12.x/controllers
11. Eloquent: Getting Started – Laravel: https://laravel.com/framework/docs/12.x/eloquent
12. Views – Laravel: https://laravel.com/framework/docs/12.x/views
13. DispatcherServlet – Spring: https://docs.spring.io/spring-framework/reference/web/webmvc/mvc-servlet.html
14. FAQ: General – Django: https://docs.djangoproject.com/en/5.2/faq/general/
15. Representational State Transfer (REST) – Roy Fielding: https://ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm
16. Angular Routing: https://angular.dev/guide/routing
17. Describir la UI – React: https://es.react.dev/learn/describing-the-ui
18. Introduction – Vue.js: https://vuejs.org/guide/introduction.html
19. Server and Client Components – Next.js: https://nextjs.org/docs/app/getting-started/server-and-client-components
20. Rendering Modes – Nuxt: https://learn.nuxt.com/en/concepts/rendering-modes
21. What is Jamstack?: https://jamstack.org/what-is-jamstack/
22. Descripción general de la API de datos de YouTube – Google for Developers: https://developers.google.com/youtube/data/getting-started?hl=es
23. YouTube Player API Reference for iframe Embeds – Google for Developers: https://developers.google.com/youtube/iframe_api_reference?hl=es-419
24. Who's using Polymer? – GitHub: https://github.com/Polymer/polymer/wiki/Who's-using-Polymer%3F
25. BeyondProd – Google Cloud: https://docs.cloud.google.com/docs/security/beyondprod?hl=es
26. Spanner in 2025: Innovations powering intelligent, multi-model AI applications – Google Cloud: https://cloud.google.com/blog/products/databases/spanner-in-2025

27. How YouTube uses Bigtable to power one of the world's largest streaming services – Google Cloud: https://cloud.google.com/blog/products/databases/youtube-runs-on-bigtable
28. Google Edge Network: https://peering.google.com/static/js/site/modules/pages/infrastructure.html
29. What Is Vitess: https://vitess.io/docs/25.0/overview/whatisvitess/
30. Vitess – GitHub: https://github.com/vitessio/vitess
31. Transactions – PostgreSQL: https://www.postgresql.org/docs/18/tutorial-transactions.html